EF Core: Subqueries

Concise lesson summary — queries inside queries, SQL translation, and highlighted key concepts.

1. Core idea

A **subquery** is a query inside another query. The **inner query** produces a value or set that the **outer query** uses to filter or shape the final result.

Mental model: Inner query → produces value/set → outer query uses it → final results.

2. Courses priced above average

```
var query = context.Courses
    .Where(c =>
        c.Price > context.Courses.Average(x => x.Price))
    .OrderBy(c => c.Price);
```

The inner query calculates **AVG(Price)**; the outer query returns courses priced above that average.

```
SELECT c.CourseId, c.Title, c.Code, c.Price
FROM Courses AS c
WHERE c.Price > (
    SELECT AVG(c0.Price)
    FROM Courses AS c0
)
ORDER BY c.Price;
```

3. Subqueries with Any()

Any() checks whether at least one matching row exists. The lesson's example returns instructors who teach at least one course:

```
var instructors = context.Instructors
    .Where(i => context.Courses
        .Any(c => c.InstructorId == i.InstructorId))
    .ToList();
```

Note: the lesson repeats this code under a heading about students with enrollments, but the code itself queries instructors and courses.

Other operators mentioned: **Count()**, **Average()**, **Max()**, and **Contains()**. These can support query patterns where one result depends on another.

4. Avoid unnecessary client-side work

Avoid: calling ToList() before Average() when the database can calculate the aggregate.

```
// Loads rows, then calculates in memory
var avg = context.Courses
    .ToList()
    .Average(c => c.Price);

// Lets the database calculate the average
var avgInDatabase = context.Courses
    .Average(c => c.Price);
```

ToList() first loads rows into application memory. Calling Average() on the EF Core query lets the database calculate the aggregate.

5. Embed the subquery in the filter

```
var courses = context.Courses
    .Where(c => c.Price >
        context.Courses.Average(x => x.Price))
    .ToList();
```

Embedding Average() in the filter allows EF Core to express the comparison as a SQL subquery, potentially avoiding a separate round trip to fetch the average first.

6. SQL inspection

ToQueryString() previews the SQL query shape. Runtime logging can show commands actually executed, which is useful for checking aggregate/subquery translation.

Final takeaway

If one query needs another query's answer, think subquery. Keep filtering and aggregation in the database when translatable, and avoid loading whole tables into memory unnecessarily.

Exact SQL depends on the LINQ expression, model, provider, and EF Core version.